기계학습특론

CHAPTER 9-2

# 의사결정나무

트리 구조 · 엔트로피와 정보이득 · ID3/C4.5/CART · 회귀나무 · 가지치기와 편향-분산 · 랜덤 포레스트와 그래디언트 부스팅 · 해석과 실무
(2026년 9월 개정판)



성명 : 최승호
이메일 : jcn99250@naver.com

기존 자료(30장)의 컴퓨터 구매 예제(ID3 손계산)와 회귀나무 예제를 유지하고, CART·앙상블·해석·최신 동향을 보강함

# 학습 목표

01

## 의사결정나무의 구조와 분류 원리

노드·가지·잎으로 이루어진 트리가 if-else 규칙으로 데이터를 분류하는 과정을 설명할 수 있다.

02

## 엔트로피와 정보이득

엔트로피·조건부 엔트로피·정보이득을 정의하고 직접 계산하여 최적 분할 속성을 선택하며, 정보이득이 상호정보량과 같음을 유도할 수 있다.

03

## 트리 성장 알고리즘과 회귀나무

ID3·C4.5·CART의 차이(이득비·지니·이진 분할·연속형 처리)와 회귀나무의 분할 기준(MSE)을 이해한다.

04

## 과적합 제어와 앙상블

가지치기(사전·비용복잡도)와 편향-분산 관점에서 트리의 한계를 이해하고, 랜덤 포레스트·그래디언트 부스팅(XGBoost/LightGBM/CatBoost)의 원리를 설명할 수 있다.

05

## 해석 · 실무 · 실습

특성 중요도와 SHAP으로 트리 모델을 해석하고, 표 형태 데이터에서의 최신 동향(TabPFN 등)을 알며 scikit-learn 1.7+로 구현할 수 있다.

# 강의 로드맵

**Part 1**
지도학습 복습 · 트리 구조 · 분류 규칙

**Part 2**
엔트로피 · 정보이득 · ID3 손계산 (14건 예제)

**Part 3**
이득비 · 지니 · CART · 연속형 · 회귀나무

**Part 4**
과적합 · 가지치기 · 이론 · 앙상블

**Part 5**
해석 · 실무 · 최신 동향 · 실습

## 기존 자료에서 유지한 부분

- 타이타닉 생존 예측 → 트리 구조 → 아키네이터 비유
- 엔트로피 정의와 컴퓨터 구매 데이터 14건 (H = 0.94)
- 나이·수입·학생·신용의 정보이득 손계산 → ID3 트리 완성
- 회귀나무: 프로야구 연차·안타 → 연봉 예제 (Step 1~3)

## 2026 개정에서 보강한 부분

- 정보이득 = 상호정보량 유도, 편향과 이득비(C4.5), 지니 불순도(CART)
- 연속형 속성 임계값 탐색, 결측·범주형 처리(sklearn 1.3+)
- 가지치기(ccp_alpha)와 편향-분산, 축 정렬 편향·NP-완전성 이론
- 랜덤 포레스트·GBM·XGBoost·LightGBM·CatBoost 수식과 비교
- SHAP(TreeSHAP)·순열 중요도, TabPFN 등 표 형태 데이터 최신 동향
- 실습: scikit-learn 1.7 + XGBoost + SHAP 노트북

# 지도학습 복습 — 분류 데이터 (타이타닉 생존 예측)

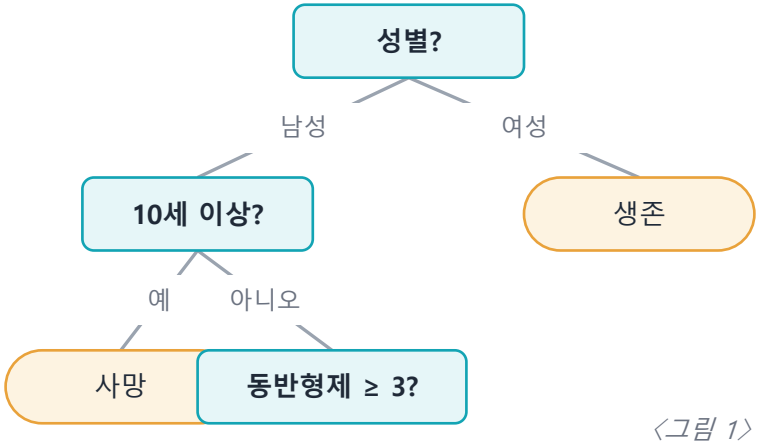
| 이름 | 나이 ($x_1$) | 동반 형제 ($x_2$) | 성별 ($x_3$) | 생존 여부 ($y$) |
|---|---|---|---|---|
| 안젤라 | 3 | 1 | 여성 | 생존 |
| 제임스 | 7 | 2 | 남성 | 생존 |
| 로널드 | 4 | 3 | 남성 | 사망 |
| 헬렌 | 33 | 3 | 여성 | 생존 |
| 오스틴 | 35 | 0 | 남성 | 사망 |
| 엔세니 | 19 | 4 | 남성 | 사망 |
| 아니스 | 29 | 0 | 남성 | 사망 |
| 디카프리오 | 8 | 3 | 남성 | ??? |

**문제 설정**

- 실제 인공지능 전공 교육에서 널리 쓰이는 데이터 (Kaggle Titanic). 생존 여부·객실 등급·성별·나이·동반 형제·요금 등으로 구성
- 특징 벡터 x = (나이, 동반 형제, 성별) 로 정답 라벨 y ∈ {생존, 사망} 을 예측하는 이진 분류 문제
- 지난주까지의 방법: KNN, 로지스틱 회귀, MLP 로도 풀 수 있음 — 그렇다면 의사결정나무는 무엇이 다른가?

**의사결정나무의 답**

- '성별이 무엇입니까?' → '10세 이상입니까?' → … 처럼 사람이 읽을 수 있는 질문의 연쇄로 예측 (규칙 자체가 모델)




*〈그림 1〉 타이타닉 생존 예측 의사결정나무 (일부)*

# 의사결정나무란 (Decision Tree)

### 정의

- 데이터에 존재하는 속성들을 나누는 일련의 규칙(질문)들을 학습을 통해 찾아 트리 형태로 표현하는 방법
- 규칙을 프로그래밍의 if–else 문으로 표현하며, 트리를 따라 내려가는 것이 곧 예측 과정
- 마지막 노드(잎)에는 분류 문제에서는 클래스(또는 클래스 확률), 회귀 문제에서는 예측값이 들어감

### 왜 배우는가

- 해석 가능성: 예측 근거를 규칙으로 설명 → 임상 결정 규칙(clinical decision rule), 규제 대응
- 전처리 불필요: 스케일링 무관(단조 변환 불변), 범주형·연속형 혼합 처리, 비선형 상호작용 자동 포착
- 앙상블의 기본 단위: 랜덤 포레스트·그래디언트 부스팅(XGBoost 등)은 표형태 데이터에서 여전히 최강 계열 (Part 4)

### 약점

- 단일 트리는 분산이 크고(데이터가 조금 바뀌면 트리가 크게 바뀜) 과적합하기 쉬움, 축 정렬 분할이라 대각선 경계에 비효율적

### 한 줄 요약

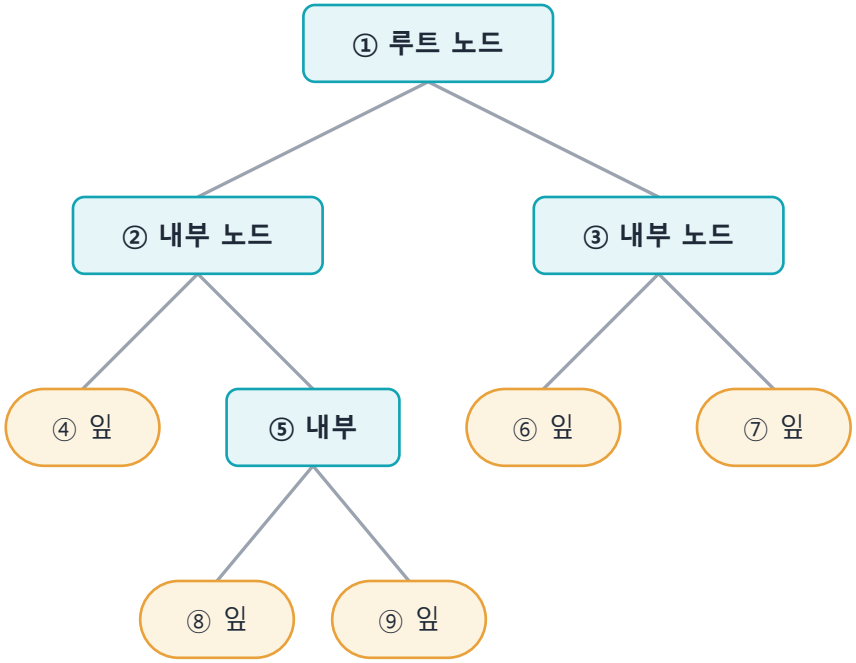
**트리 = '입력 공간을 축에 평행한 사각형 영역으로 재귀 분할하고, 각 영역에 상수 예측을 부여하는 모델'**

$$f(x) = \sum_m c_m \cdot \mathbb{1}[x \in R_m]$$

### 역사

- ID3 (Quinlan 1986): 정보이득, 다지 분할
- C4.5 (Quinlan 1993): 이득비, 연속형, 결측, 가지치기
- CART (Breiman et al. 1984): 지니, 이진 분할, 회귀나무 — scikit-learn 의 구현
- CHAID: $\chi^2$ 검정 기반 (통계학·마케팅)

# 트리 구조의 용어




*〈그림 2〉 트리 구조의 예시 — 깊이(depth) = 3*

| 용어 | 정의 |
|---|---|
| 노드 (Node) | 질문(분할) 또는 예측이 놓이는 지점 |
| 루트 노드 | 시작 노드. 전체 데이터를 포함 (①) |
| 부모 / 자식 노드 | 주어진 노드의 상위 / 하위 노드 (② 의 자식은 ④·⑤) |
| 형제 노드 | 부모가 같은 노드 (④ 와 ⑤) |
| 잎 노드 (Leaf) | 자식이 없는 노드 = 예측이 놓이는 노드 (④⑥⑦⑧⑨) |
| 엣지 / 가지 | 노드와 노드를 연결하는 간선 = 질문의 답 |
| 깊이 (Depth) | 루트에서 잎까지의 최대 간선 수 — 과적합 제어의 핵심 초매개변수 |

이진 트리(CART)에서 잎이 L개면 내부 노드는 L−1개. 잎 수 L 은 '영역의 수' 이자 모델 복잡도 — 가지치기 목적함수(Part 4)에서 벌점 항으로 등장한다. 깊이 d 인 완전 이진 트리의 잎은 최대 $2^d$ 개.

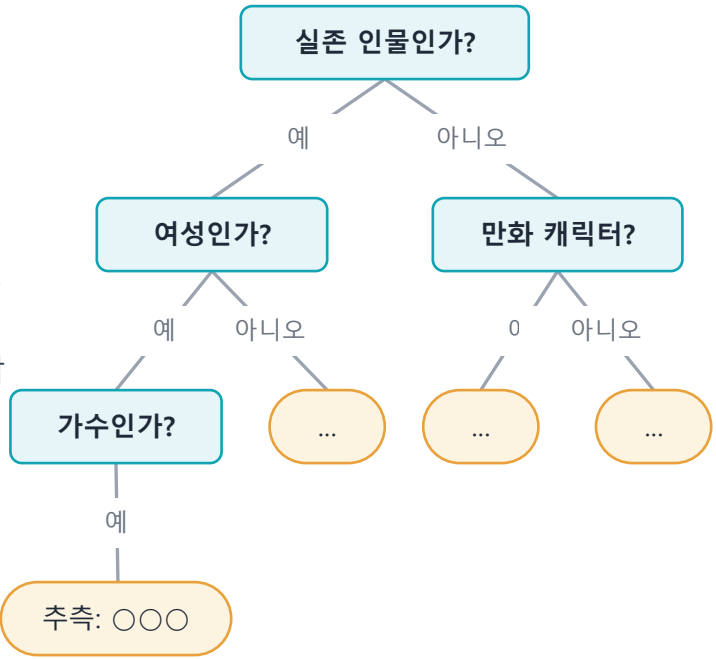
# 의사결정나무 예시 — 아키네이터와 스무고개

**아키네이터 (Akinator) 게임**

- 사용자가 떠올린 인물을 '실존 인물인가?', '여성인가?', '가수인가?' 같은 예/아니오 질문 20개 안팎으로 맞히는 웹 게임 — 스무고개 놀이와 같은 구조
- 각 질문 = 내부 노드, 답 = 가지, 최종 추측 = 잎. 수천 명의 인물을 20개 질문으로 구분하려면 질문 하나가 후보를 '절반씩' 줄여야 함 ($2^{20} \approx 10^6$)

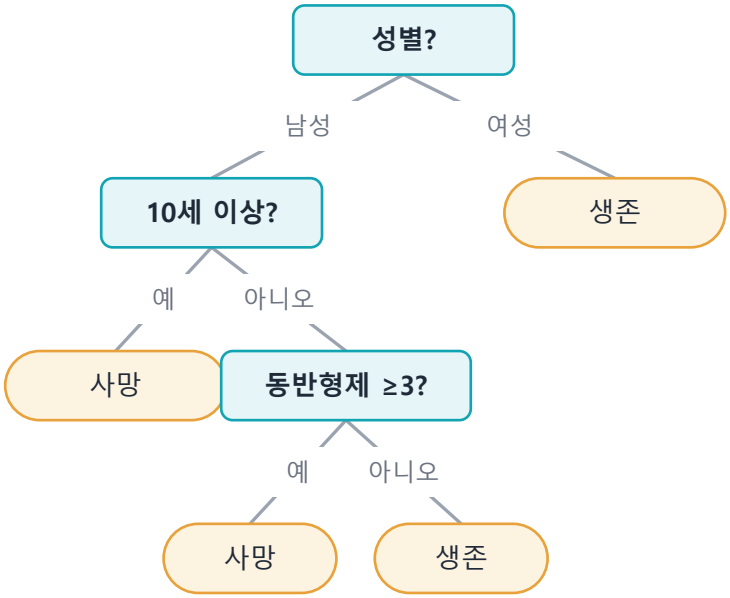
**핵심 통찰: 좋은 질문이란?**

- 답을 들었을 때 후보가 가장 크게 줄어드는 질문 = 불확실성(엔트로피)을 가장 많이 줄이는 질문
- '그 사람은 나폴레옹인가?' 는 거의 항상 '아니오' → 정보가 거의 없음. '남성인가?' 는 후보를 절반으로 → 정보가 큼
- 이 직관을 수식으로 만든 것이 정보이득(information gain) — Part 2

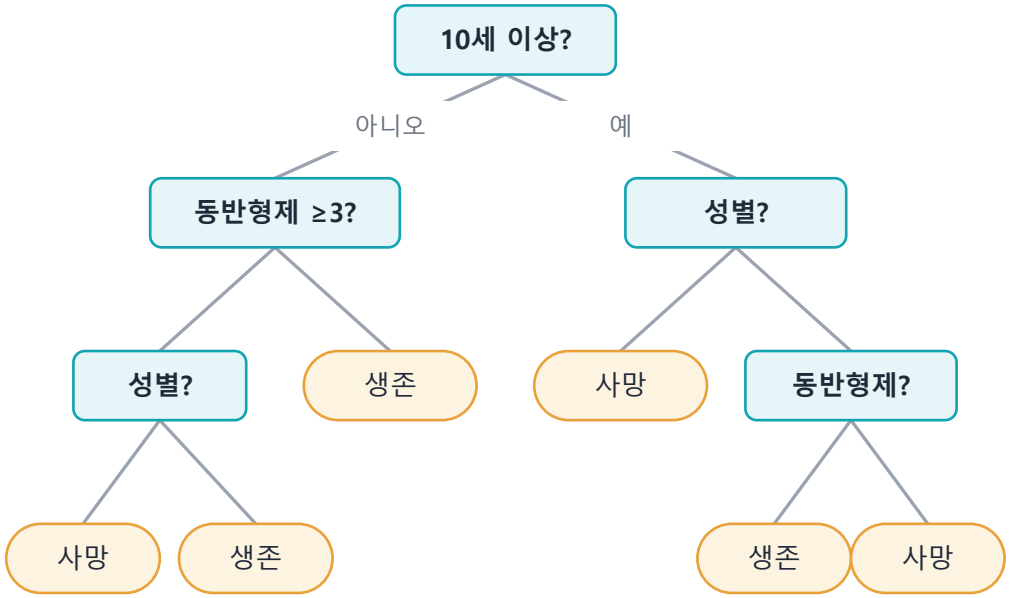



*〈그림 3〉 스무고개 = 의사결정나무*

# 의사결정나무를 통한 분류 — 분할 속성 선택이 핵심




*(a) 성별을 먼저 묻는 트리 — 깊이 3, 잎 4개*

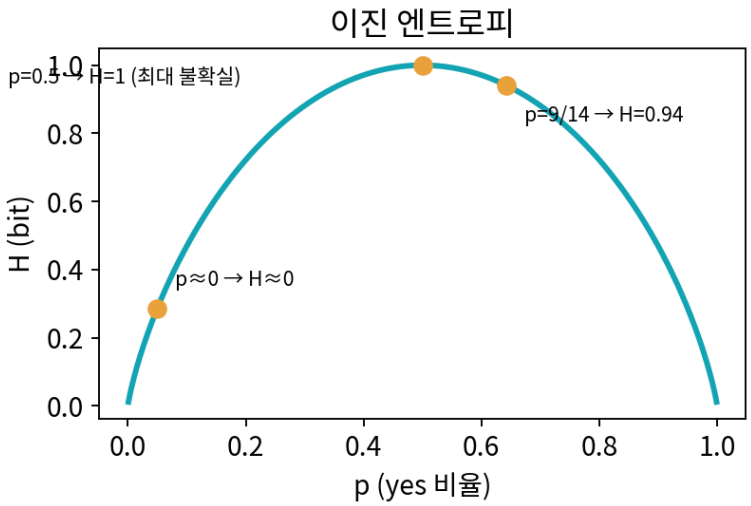



*(b) 나이를 먼저 묻는 트리 — 깊이 3, 잎 6개 (더 복잡)*

- 의사결정나무의 노드 구성(분류 규칙 결정)이 가장 중요 — 같은 데이터를 같은 정확도로 분류하는 트리라도 어떤 속성을 먼저 묻느냐에 따라 크기가 달라짐
- 분할 속성(splitting attribute): 부모 노드에 들어가는 if–else 조건. 각 노드에서 '어떤 속성이 가장 효과적으로 클래스를 나누는가' 를 계산해 결정 → 정보이득
- 작은 트리를 선호하는 이유 (오컴의 면도날): 해석이 쉽고, 같은 훈련 정확도라면 일반화 오차가 작을 가능성이 높음 (Part 4 이론)

# 엔트로피의 이해

$$H(D) = -\Sigma_{i=1}^{m} p_i \log_2 p_i \qquad (D: \text{데이터 집합}, m: \text{클래스 수}, p_i: \text{클래스 } i \text{ 의 비율}, \ 0 \cdot \log 0 \equiv 0)$$




**직관**

- 데이터 집합이 제공하는 '각 사건이 일어날 확률에 기반한 정보량의 기댓값' (Shannon 1948). 단위 bit ($\log_2$)
- 낮은 엔트로피 = 경우의 수가 적음 = 낮은 정보량 = 낮은 불확실성 / 높은 엔트로피 = 클래스가 골고루 섞여 있음
- 이진 분류: p = 0.5 에서 최대 1 bit, p → 0 또는 1 에서 0 (순수 노드)

**성질**

- $H \geq 0$, 균등분포에서 최대 $\log_2 m$, 오목(concave) 함수 — 오목성이 정보이득 ≥ 0 을 보장 (뒤 슬라이드)
- 부호화 해석: 클래스를 전송할 때 필요한 평균 최소 비트 수 (Shannon 소스 부호화 정리)

**의사결정나무의 아이디어: 어떤 노드에서 분할 속성을 선택했을 때 자식 노드들의 엔트로피가 낮아지도록(=클래스가 순수해지도록) 속성을 고르고, 잎에서는 엔트로피가 0 에 가깝게 만든다.**

# 엔트로피의 예시 — 컴퓨터 구매 여부 데이터

| 번호 | 나이 ($x_1$) | 수입 ($x_2$) | 학생 ($x_3$) | 신용 ($x_4$) | 구매 (y) |
|---|---|---|---|---|---|
| 1 | 청소년 | 고소득 | 아니오 | 좋음 | 미구매 |
| 2 | 청소년 | 고소득 | 아니오 | 아주좋음 | 미구매 |
| 3 | 청년 | 고소득 | 아니오 | 좋음 | 구매 |
| 4 | 중년 | 중소득 | 아니오 | 좋음 | 구매 |
| 5 | 중년 | 저소득 | 예 | 좋음 | 구매 |
| 6 | 중년 | 저소득 | 예 | 아주좋음 | 미구매 |
| 7 | 청년 | 저소득 | 예 | 아주좋음 | 구매 |
| 8 | 청소년 | 중소득 | 아니오 | 좋음 | 미구매 |
| 9 | 청소년 | 저소득 | 예 | 좋음 | 구매 |
| 10 | 중년 | 중소득 | 예 | 좋음 | 구매 |
| 11 | 청소년 | 중소득 | 예 | 아주좋음 | 구매 |
| 12 | 청년 | 중소득 | 아니오 | 아주좋음 | 구매 |
| 13 | 청년 | 고소득 | 예 | 좋음 | 구매 |
| 14 | 중년 | 중소득 | 아니오 | 아주좋음 | 미구매 |

**데이터 (Han & Kamber, AllElectronics)**

- 사람의 나이·수입·학생 여부·신용 등급을 고려하여 컴퓨터 구매 여부를 예측 (n = 14, 이진 분류)
- 구매 = 9건 → p(구매) = 9/14, 미구매 = 5건 → p(미구매) = 5/14

$$H(D) = -(9/14)\cdot\log_2(9/14) - (5/14)\cdot\log_2(5/14)$$
$$= 0.410 + 0.531 = 0.940 \text{ bit}$$

- 0.94 는 최댓값 1 에 가까움 → 이 데이터는 클래스가 꽤 섞여 있어 '아직 예측하기 어렵다'
- 목표: 어떤 속성으로 데이터를 나누면 나눈 뒤의 (가중) 엔트로피가 가장 작아지는가? → 정보이득
- *실습에서는 np.log2 로 계산. p = 0 인 항은 0·log 0 = 0 으로 처리해야 함 (p[p > 0] 로 필터링)*

# 정보이득 (Information Gain)

$$\mathrm{Gain}(D, A) = H(D) - H_A(D), \quad H_A(D) = \Sigma_\{v \in \mathrm{Values}(A)\} \, (|D_v| \,/\, |D|) \cdot H(D_v)$$

**정의**

- 학습 데이터를 특정 속성 A 로 분할했을 때 각 속성값의 효율성에 대한 측도 — 분할 전 엔트로피에서 분할 후 자식 노드들의 가중 평균 엔트로피(조건부 엔트로피 H(D|A))를 뺀 값
- H_A(D): 속성 A 로 분할했을 때 A 의 각 값 v 를 갖는 부분집합 $D_v$ 의 엔트로피를 크기 비율 $|D_v|/|D|$ 로 가중합
- 각 노드에서 모든 후보 속성의 정보이득을 계산하고 가장 큰 속성을 분할 속성으로 선택 → 자식 노드에서 재귀 (ID3)

**해석**

- Gain 이 클수록 '그 속성을 알면 클래스에 대한 불확실성이 많이 줄어든다' — 스무고개의 좋은 질문
- Gain = 0 이면 속성과 클래스가 (경험적으로) 독립 → 나눌 이유 없음

## 표기 정리

Values(A): 속성 A 가 가질 수 있는 값의 집합 (예: 나이 = {청소년, 청년, 중년})

$D_v$: A = v 인 데이터의 부분집합

|D|: 데이터 수 (=14)

H_A(D) 를 정보론에서는 조건부 엔트로피 H(Y | A) 라고 씀

**따라서 Gain(D, A) = H(Y) − H(Y|A) = I(Y; A) — 상호정보량!**

# 이론 (1) — 정보이득은 상호정보량이며 항상 ≥ 0

**정리**

$$\mathrm{Gain}(D, A) = H(Y) - H(Y \mid A) = I(Y; A) = \sum_{a,y} p(a, y) \log \frac{p(a, y)}{p(a)\,p(y)} \geq 0,$$ 등호는 $Y \perp\!\!\!\perp A$ 일 때

**증명 스케치**

- $H(Y|A) = \sum_a p(a)\, H(Y|A=a)$ 는 정의상 정보이득의 두 번째 항(가중 엔트로피)과 같다. 연쇄 법칙 $H(Y|A) = H(A, Y) - H(A)$ 를 대입하면 $I(Y;A) = H(Y) + H(A) - H(A,Y)$.
- $I(Y;A) = KL(\, p(a,y) \,\|\, p(a)p(y)\,)$ 이고 KL 발산은 Gibbs 부등식(또는 log 의 오목성에 대한 Jensen 부등식)에 의해 항상 ≥ 0. 따라서 어떤 속성으로 나누어도 가중 엔트로피는 증가하지 않는다. ■
- 동치 해석: H 가 오목 함수이므로 $H(\sum p_v\, q_v) \geq \sum p_v\, H(q_v)$ — 부분집합들의 클래스 분포를 섞은 것(부모)의 엔트로피는 부분집합 엔트로피의 평균보다 크거나 같다.

**함의**

- 정보이득이 음수가 될 수 없으므로 탐욕적 성장은 '멈출 이유' 를 스스로 만들지 못한다 → 잎이 순수해질 때까지 자라 과적합. 그래서 정지 조건·가지치기가 필요 (Part 4)
- 정보이득은 값이 많은 속성(예: ID, 날짜)에 편향된다: 각 값이 데이터 1개씩이면 $H_A(D) = 0$ 이 되어 Gain 이 최대가 되지만 일반화에는 쓸모없음 → 이득비 (Part 3)
- 지니 불순도로 바꿔도 같은 부등식이 성립(지니도 오목) — 지니 감소량 역시 항상 ≥ 0

# 의사결정나무의 성장 — ID3 알고리즘

```
ID3(D, 속성 집합 A):
    if D 의 모든 샘플이 같은 클래스 c:          return 잎(c)
    if A 가 비어 있음 (더 나눌 속성 없음):       return 잎(D 의 다수 클래스)
    A* ← argmax_{A ∈ 속성집합} Gain(D, A)          # 정보이득이 최대인 속성 선택 (탐욕)
    노드 ← 내부노드(질문 = A*)
    for v in Values(A*):
        D_v ← { (x, y) ∈ D : x[A*] = v }
        if D_v 비어 있음:  자식 ← 잎(D 의 다수 클래스)
        else:              자식 ← ID3(D_v, 속성집합 - {A*})   # 재귀
    return 노드
```

**성장(grow)**

- 일반적으로 의사결정나무를 생성하는 방법. 루트에서 시작해 각 노드에서 정보이득이 가장 큰 속성으로 데이터를 반복적(iteratively)으로 나누는(divides) 하향식(top-down) 탐욕(greedy) 알고리즘 — ID3 = Iterative Dichotomiser 3 (Quinlan 1986)
- 각 노드마다 그 노드에 도달한 데이터 부분집합으로 정보이득을 다시 계산 → 같은 속성이라도 노드마다 정보이득이 다름

**예제에서 계산할 것**

- 루트에서 Gain(D, 나이), Gain(D, 수입), Gain(D, 학생), Gain(D, 신용) 을 각각 계산 → 최대 선택 → 자식 노드에서 반복

$$\text{Gain}(D, \text{나이}) = H(D) - H_{\text{나이}}(D) \quad \text{Gain}(D, \text{수입}) = H(D) - H_{\text{수입}}(D) \quad \text{Gain}(D, \text{학생}) = H(D) - H_{\text{학생}}(D) \quad \text{Gain}(D, \text{신용}) = H(D) - H_{\text{신용}}(D)$$

# 트리의 성장 (1) — 나이 기준의 정보이득

| 번호 | 나이 ($x_1$) | 수입 ($x_2$) | 학생 ($x_3$) | 신용 ($x_4$) | 구매 (y) |
|---|---|---|---|---|---|
| 1 | 청소년 | 고소득 | 아니오 | 좋음 | 미구매 |
| 2 | 청소년 | 고소득 | 아니오 | 아주좋음 | 미구매 |
| 3 | 청년 | 고소득 | 아니오 | 좋음 | 구매 |
| 4 | 중년 | 중소득 | 아니오 | 좋음 | 구매 |
| 5 | 중년 | 저소득 | 예 | 좋음 | 구매 |
| 6 | 중년 | 저소득 | 예 | 아주좋음 | 미구매 |
| 7 | 청년 | 저소득 | 예 | 아주좋음 | 구매 |
| 8 | 청소년 | 중소득 | 아니오 | 좋음 | 미구매 |
| 9 | 청소년 | 저소득 | 예 | 좋음 | 구매 |
| 10 | 중년 | 중소득 | 예 | 좋음 | 구매 |
| 11 | 청소년 | 중소득 | 예 | 아주좋음 | 구매 |
| 12 | 청년 | 중소득 | 아니오 | 아주좋음 | 구매 |
| 13 | 청년 | 고소득 | 예 | 좋음 | 구매 |
| 14 | 중년 | 중소득 | 아니오 | 아주좋음 | 미구매 |

*〈표 2〉 컴퓨터 구매 여부 데이터 예시 (색: 나이별 그룹)*

**나이 속성에 부여 가능한 값: 청소년, 청년, 중년**

- 청소년 (5건): 구매 2, 미구매 3 → $H = -(2/5)\log_2(2/5) - (3/5)\log_2(3/5) = 0.971$
- 청년 (4건): 구매 4, 미구매 0 → $H = 0$ (순수!)
- 중년 (5건): 구매 3, 미구매 2 → $H = 0.971$

$$H_나이(D) = (5/14)\cdot 0.971 + (4/14)\cdot 0 + (5/14)\cdot 0.971 = 0.694$$

$$\text{Gain}(D, 나이) = 0.940 - 0.694 = 0.247$$

- 청년 그룹은 4명 전원이 구매 → 이 가지는 바로 잎이 됨
- 나머지 두 그룹은 여전히 섞여 있음(0.971) → 자식 노드에서 추가 분할 필요

# 트리의 성장 (2) — 수입 · 학생 · 신용 기준의 정보이득

| 속성 | 값별 (구매, 미구매) 분포 | 값별 엔트로피 | 가중 엔트로피 H_A(D) | Gain(D, A) |
|---|---|---|---|---|
| 나이 | 청소년 (2,3) · 청년 (4,0) · 중년 (3,2) | 0.971 · 0 · 0.971 | 0.694 | 0.247 ★ |
| 수입 | 고소득 (2,2) · 중소득 (4,2) · 저소득 (3,1) | 1.000 · 0.918 · 0.811 | (4/14)·1 + (6/14)·0.918 + (4/14)·0.811 = 0.911 | 0.029 |
| 학생 | 예 (6,1) · 아니오 (3,4) | 0.592 · 0.985 | (7/14)·0.592 + (7/14)·0.985 = 0.789 | 0.151 |
| 신용 | 좋음 (6,2) · 아주좋음 (3,3) | 0.811 · 1.000 | (8/14)·0.811 + (6/14)·1 = 0.892 | 0.048 |

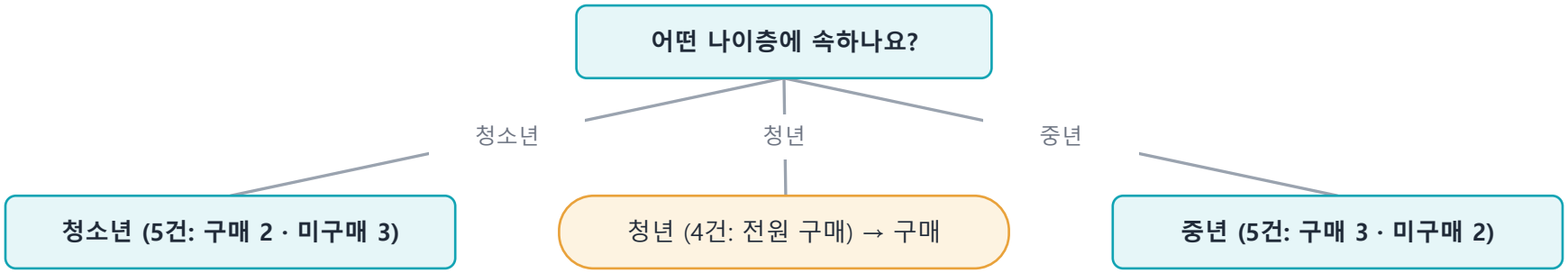
**결론: 첫 번째 분할 속성 = 나이 (Gain 0.247 최대)**

- 수입은 0.029 로 거의 정보가 없음 — 수입만 알아서는 구매 여부를 거의 예측할 수 없다는 뜻
- 학생 여부(0.151)가 두 번째로 유용 — 루트에서는 선택되지 않았지만 자식 노드에서 다시 후보가 됨

**계산 팁**

- 이진 엔트로피 값 암기: (1,1) → 1.000, (2,1) → 0.918, (3,1) → 0.811, (3,2) → 0.971, (6,1) → 0.592, (4,3) → 0.985
- 실습 노트북의 get_info() / get_attribute_info() 함수가 이 표를 자동 생성

# 트리의 성장 (3) — 나이 기준으로 데이터 분할




| 번호 | 나이 ($x_1$) | 수입 ($x_2$) | 학생 ($x_3$) | 신용 ($x_4$) | 구매 (y) |
|---|---|---|---|---|---|
| 1 | 청소년 | 고소득 | 아니오 | 좋음 | 미구매 |
| 2 | 청소년 | 고소득 | 아니오 | 아주좋음 | 미구매 |
| 8 | 청소년 | 중소득 | 아니오 | 좋음 | 미구매 |
| 9 | 청소년 | 저소득 | 예 | 좋음 | 구매 |
| 11 | 청소년 | 중소득 | 예 | 아주좋음 | 구매 |

*〈표 3〉 청소년 데이터 — 추가 분할 필요*

| 번호 | 나이 ($x_1$) | 수입 ($x_2$) | 학생 ($x_3$) | 신용 ($x_4$) | 구매 (y) |
|---|---|---|---|---|---|
| 4 | 중년 | 중소득 | 아니오 | 좋음 | 구매 |
| 5 | 중년 | 저소득 | 예 | 좋음 | 구매 |
| 6 | 중년 | 저소득 | 예 | 아주좋음 | 미구매 |
| 10 | 중년 | 중소득 | 예 | 좋음 | 구매 |
| 14 | 중년 | 중소득 | 아니오 | 아주좋음 | 미구매 |

*〈표 4〉 중년 데이터 — 추가 분할 필요*

- 청년 가지는 잎으로 종료. 청소년·중년 가지에서는 남은 속성 {수입, 학생, 신용} 으로 정보이득을 '해당 부분집합에 대해' 다시 계산
- 이때 H(D_청소년) = H(D_중년) = 0.971 이 새 기준 엔트로피가 됨

# 트리의 성장 (4) — 두 번째 분할 속성 (청소년 · 중년)

### 청소년 부분집합 (구매 2, 미구매 3), H = 0.971

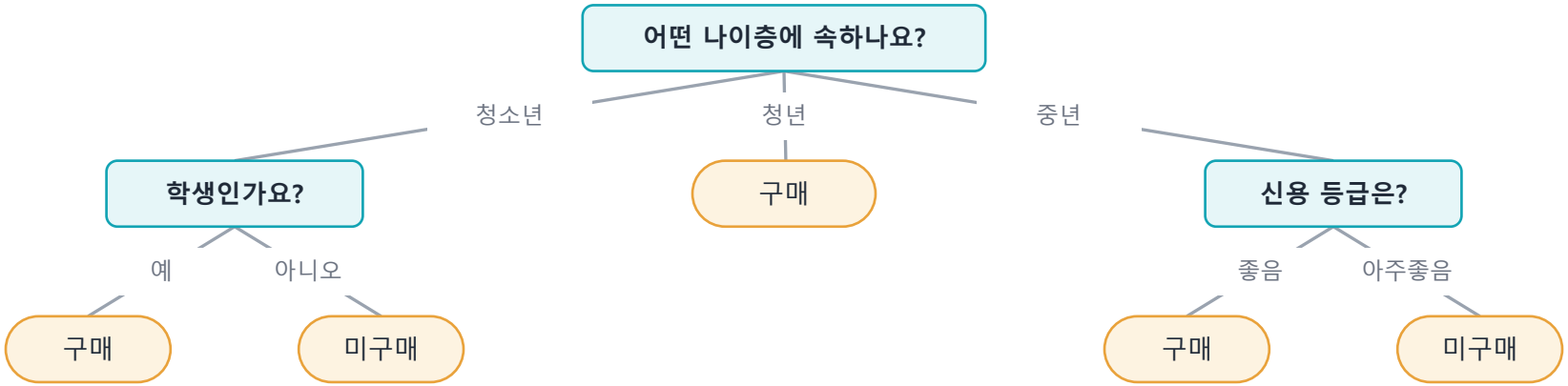
- 학생: 예 (2,0) · 아니오 (0,3) → H = 0 · 0 → Gain = 0.971 ★
- 수입: 고 (0,2) · 중 (1,1) · 저 (1,0) → (2/5)·0 + (2/5)·1 + (1/5)·0 = 0.4 → Gain = 0.571
- 신용: 좋음 (1,2) · 아주좋음 (1,1) → (3/5)·0.918 + (2/5)·1 = 0.951 → Gain = 0.020

**→ 학생 여부로 분할하면 두 자식 모두 순수 (완벽 분리)**

### 중년 부분집합 (구매 3, 미구매 2), H = 0.971

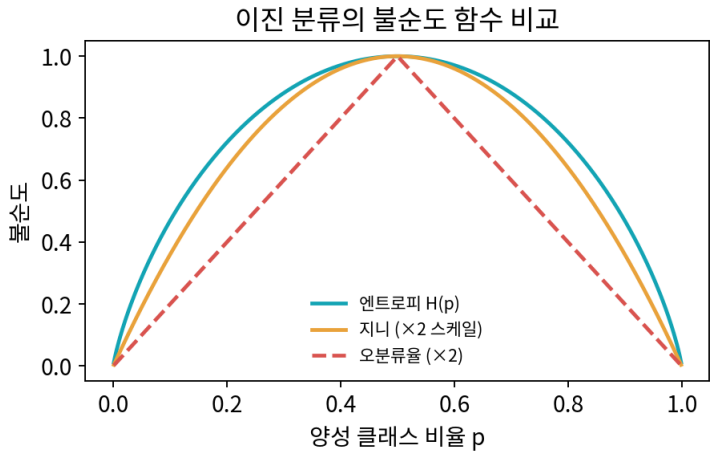
- 신용: 좋음 (3,0) · 아주좋음 (0,2) → Gain = 0.971 ★
- 학생: 예 (2,1) · 아니오 (1,1) → (3/5)·0.918 + (2/5)·1 = 0.951 → Gain = 0.020
- 수입: 중 (2,1) · 저 (1,1) → 0.951 → Gain = 0.020

**→ 신용 등급으로 분할하면 두 자식 모두 순수**




*〈그림 4〉 ID3 로 완성한 컴퓨터 구매 여부 의사결정나무 (훈련 정확도 100%, 잎 5개)*

# 정보이득의 편향과 이득비 (C4.5) · 지니 불순도 (CART)




## 이득비 (Gain Ratio, C4.5)

SplitInfo(D, A) = $-\Sigma_v$ ($|D_v|/|D|$) $\log_2$ ($|D_v|/|D|$) (속성값 분포의 엔트로피)

**GainRatio = Gain(D, A) / SplitInfo(D, A)**

- 값이 많은 속성은 SplitInfo 가 커서 벌점 → ID 같은 속성에 대한 편향 완화(완전 차단은 아님)
- 예: 번호(RID) 로 나누면 Gain = 0.94 이지만 SplitInfo = $\log_2 14$ = 3.81 → 이득비 0.25 로 크게 낮아짐 (나이의 이득비는 0.247/1.577 = 0.156). C4.5 는 Gain 이 평균 이상인 속성 중 이득비 최대를 선택

## 지니 불순도 (Gini, CART)

**Gini(D) = $1 - \Sigma_i p_i^2 = \Sigma_i p_i (1 - p_i)$**

- 무작위로 뽑은 두 샘플의 클래스가 다를 확률. 이진: 2p(1−p), 최대 0.5
- log 가 없어 계산이 빠르고, 엔트로피와 순위가 거의 같음 (그림 ). scikit-learn 기본 criterion='gini'
- 분할 기준: ΔGini = Gini(D) − $\Sigma_v$ ($|D_v|/|D|$) Gini($D_v$) 최대

## 오분류율은 왜 안 쓰나

- 오분류율 $1 - \max_i p_i$ 는 조각별 선형 → 분할 후 다수 클래스가 안 바뀌면 감소량 0 이 되어 유용한 분할을 놓침
- 엔트로피·지니는 엄격하게 오목 → 클래스 분포가 조금만 순수해져도 보상
- 실무: 지니 vs 엔트로피 차이는 미미(≈2% 경우만 다른 트리, Raileanu & Stoffel 2004). 속도 때문에 지니가 기본

# CART — 이진 분할, 연속형 속성, 결측치

**CART (Classification And Regression Trees, Breiman et al. 1984) — scikit-learn 의 구현**

- 항상 이진 분할: 범주형도 '{청소년} vs {청년, 중년}' 처럼 두 집합으로 나눔 → 다지 분할보다 데이터 단편화(fragmentation)가 덜 함
- 분류: 지니(또는 엔트로피, log_loss), 회귀: MSE(또는 MAE, Poisson, Friedman-MSE)

**연속형 속성의 임계값 탐색**

- 속성값을 정렬(O(n log n)) 후 인접한 두 값의 중점 $x_i + (x_{i+1} - x_i)/2$ 를 후보 임계값으로 → 각 후보에서 불순도 감소를 계산(누적합으로 O(n)). 노드당 O(d · n log n)
- 단조 변환(로그, 표준화)에 불변 — 임계값만 바뀔 뿐 트리는 동일 → 트리 모델에 스케일링이 필요 없는 이유

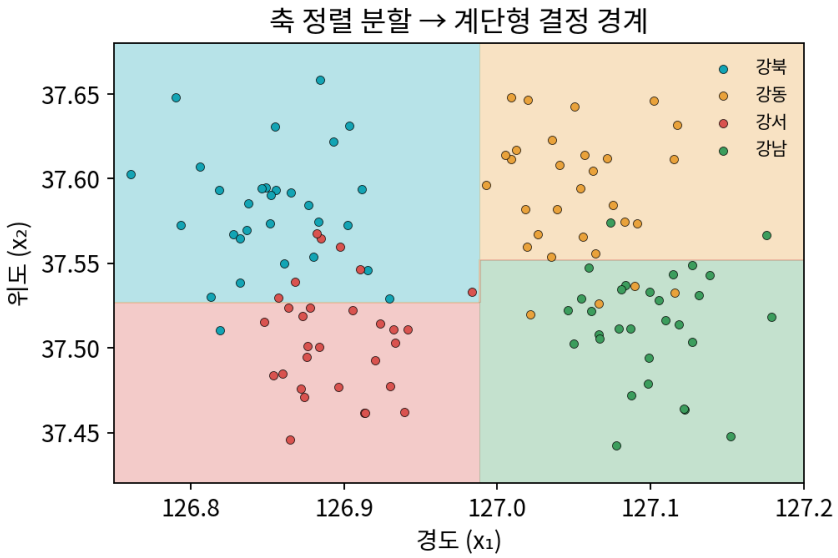
**결측치 처리**

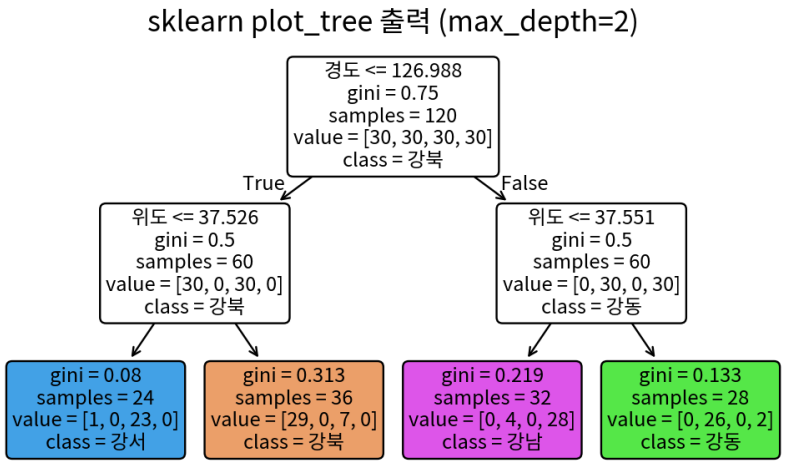
- C4.5: 결측 샘플을 가지별로 비율 분배 (fractional instances). CART: 대리 분할(surrogate split) — 주 속성과 가장 비슷하게 나누는 다른 속성 사용
- scikit-learn ≥ 1.3: DecisionTree 가 NaN 을 직접 지원 (RandomForest·ExtraTrees 는 1.4 부터) — 학습 시 결측 샘플을 왼쪽/오른쪽 중 불순도가 낮은 쪽으로 보내는 규칙을 학습 (HistGradientBoosting 은 1.0 부터)

**범주형 속성**

- scikit-learn 트리는 원-핫 또는 순서 인코딩 필요 (HistGradientBoosting 은 categorical_features 로 네이티브 지원). LightGBM·CatBoost 는 범주형을 직접 처리

# 연속형 속성의 분할 — 서울시 구 분류 예시 (실습 데이터)




**데이터**

- 서울시 각 구·동의 경도($x_1$)·위도($x_2$) 로 강북 / 강동 / 강서 / 강남 4개 권역을 분류 (실습 노트북의 district_dict_list.csv, dong_dict_list.csv)

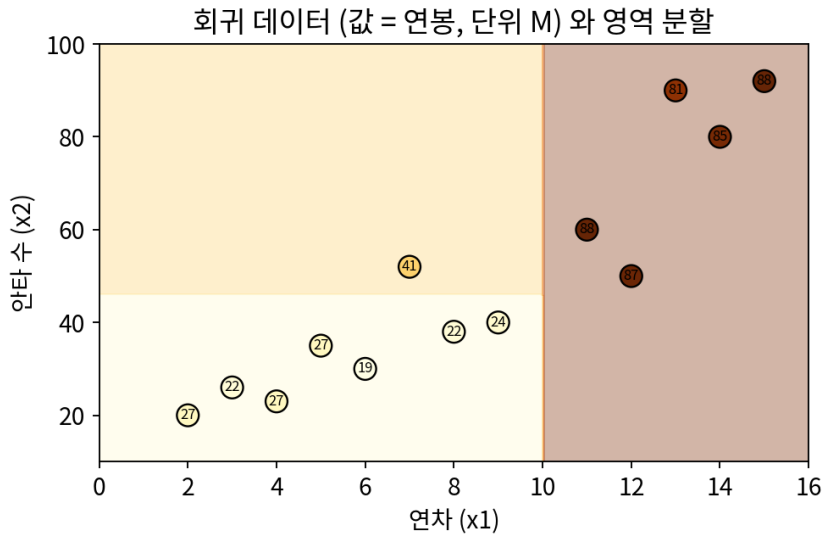
**관찰 1 — 계단형 결정 경계**

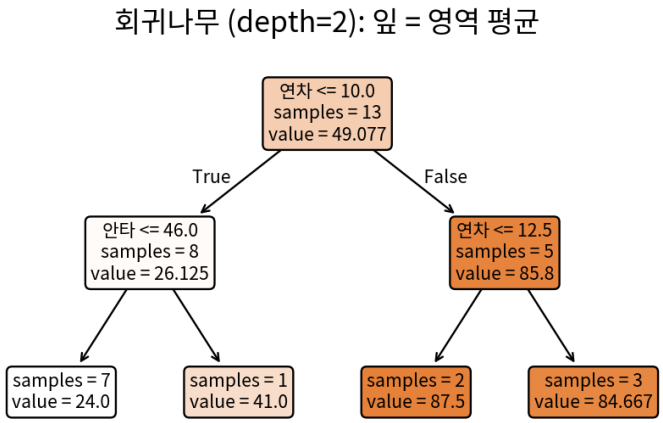
- 분할이 '경도 ≤ 126.99', '위도 ≤ 37.55' 처럼 한 축의 임계값이므로 경계가 축에 평행한 직사각형으로만 만들어짐 (축 정렬 편향)
- 연속형 속성에서는 정렬된 값들의 중점이 후보 임계값 — 여기서는 실제 한강·행정 경계와 가까운 위도·경도가 자동으로 선택됨 (해석 가능성의 장점)

**관찰 2 — plot_tree 읽는 법**

- 각 노드: 분할 조건 / 지니 / samples / value(클래스별 개수) / class(다수 클래스). 색은 다수 클래스, 진하기는 순도(지니가 낮을수록 진함)
- 여기서는 깊이 2 로 제한해 잎이 4개 — 깊이를 풀면 훈련 정확도는 100% 가 되지만 동 단위 테스트 데이터에서 성능이 떨어짐 (Part 4 과적합)

# 회귀나무 (Regression Tree)




**예제: 프로야구 선수의 연차($x_1$)·안타 수($x_2$) 로 연봉(y, 단위 M) 예측 — 기존 자료 Step 1~3**

- Step 1. 데이터 분할: 영역을 나누고 분할 기준을 확인 (연차 10년 · 안타 46 등)
- Step 2. 영역별 평균값 계산: 각 잎(영역)의 예측값 = 그 영역 관측치의 평균 (예: 24M, 41M, 87.5M, 84.7M)
- Step 3. 분할 기준과 영역 평균으로 회귀나무 생성 → 새 선수(연차 5, 안타 35)는 왼쪽 아래 영역 → 24M 예측

**분할 기준 (연속형 y)**

- 엔트로피 대신 잔차제곱합: 분할 (j, t) 는 $\min_{j,t} [ \sum_{x_i \in R_1(j,t)} (y_i - \bar{y}_1)^2 + \sum_{x_i \in R_2(j,t)} (y_i - \bar{y}_2)^2 ]$ — 영역 내 분산이 최소가 되도록. 잎의 최적 상수는 평균(제곱손실) 또는 중앙값(절대손실)

# 회귀나무 — 예측 함수의 형태와 한계

$$\hat{f}(x) = \Sigma_{m=1}^{M} \hat{c}_m \cdot \mathbb{1}[x \in R_m], \quad \hat{c}_m = \text{mean}\{ y_i : x_i \in R_m \}$$

**성질**

- 조각별 상수(piecewise constant) 함수 — 영역 경계에서 불연속, 영역 안에서는 평평. 매끄러운 함수를 근사하려면 많은 잎이 필요 → 앙상블(부스팅)이 이를 보완
- 외삽 불가: 훈련 데이터 범위 밖의 x 에는 가장자리 영역의 상수를 반환 (선형회귀와 결정적 차이 — 시계열 추세 예측에 부적합)
- 최적 분할 탐색은 각 노드에서 모든 (속성 j, 임계값 t) 쌍을 순회 — 탐욕적이므로 전역 최적 트리는 아님

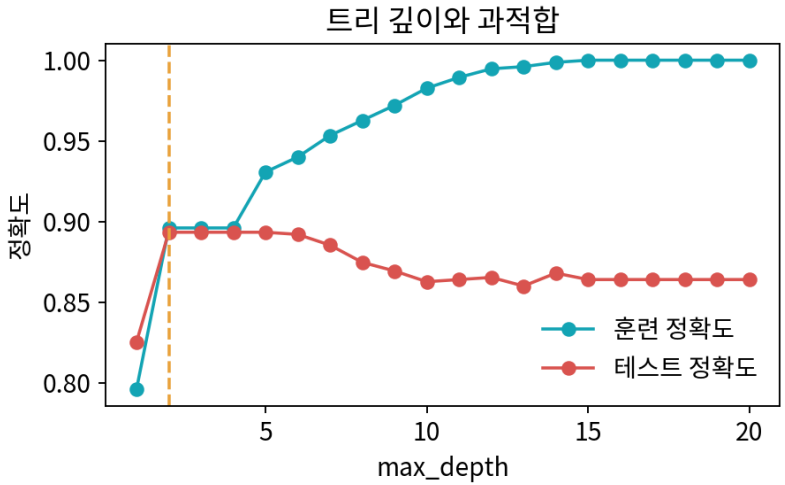
**확장**

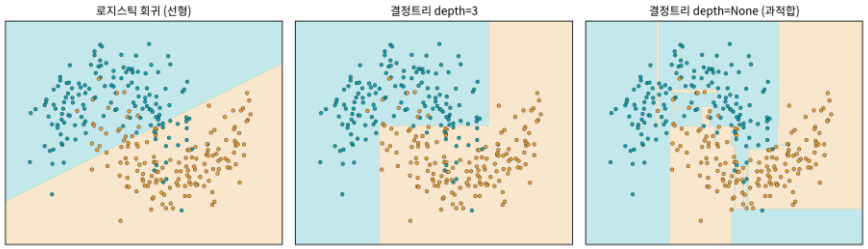
- 모델 트리(M5, Quinlan 1992): 잎에 상수 대신 선형회귀를 둠 → 매끄러운 근사, 외삽 일부 가능. scikit-learn 에는 없고 LightGBM 의 linear_tree 옵션이 유사
- Poisson / Gamma / 분위수 손실: 계수·비용·분위수 예측 (scikit-learn criterion='poisson', HistGradientBoostingRegressor(loss='quantile'))
- 다중 출력 회귀: y 가 벡터여도 잎에 평균 벡터를 두면 됨 (DecisionTreeRegressor 는 multi-output 지원)

**의료 예**

- 재원일수·의료비 예측(오른쪽 꼬리가 긴 분포 → 로그 변환 또는 Gamma/Tweedie 손실), 연속형 위험 점수 산출

# 과적합과 트리의 편향–분산



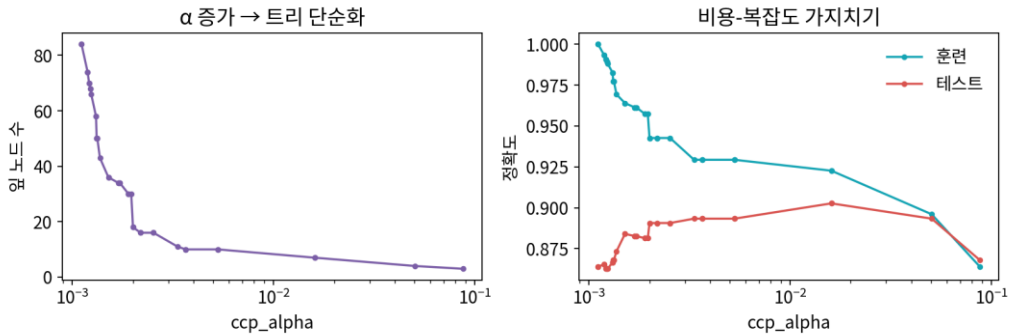

**관찰**

- 왼쪽: 깊이가 커질수록 훈련 정확도는 1.0 으로 가지만 테스트 정확도는 깊이 3~5 이후 하락 → 과적합
- 오른쪽: depth=None 트리는 잡음 점 하나를 위해 작은 사각형 영역을 만들어 냄 (계단형 경계의 기억)

**편향–분산 관점**

- 얕은 트리 = 높은 편향(단순), 낮은 분산. 깊은 트리 = 낮은 편향, 매우 높은 분산 — 데이터를 조금만 바꿔도(부트스트랩) 상위 분할부터 달라져 트리 전체가 바뀜
- 트리의 분산이 크다는 사실이 바로 배깅(랜덤 포레스트)이 효과적인 이유: 분산 감소가 목표. 부스팅은 반대로 얕은 트리(높은 편향)를 순차적으로 더해 편향을 줄임

# 가지치기 (Pruning) — 사전 · 사후




## 사전 가지치기 (early stopping)

- max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes, min_impurity_decrease
- 빠르지만 근시안적: 당장 이득이 작아도 다음 분할에서 큰 이득이 나올 수 있음 (XOR 문제)

$$R_\alpha(T) = R(T) + \alpha \cdot |\text{leaves}(T)| \quad (R: \text{훈련 오차 또는 불순도 합}, \alpha \geq 0: \text{복잡도 벌점})$$

**비용–복잡도 가지치기 (CCP, Breiman 1984)**

- 완전히 자란 트리 $T_0$ 에서 시작해, 내부 노드 t 를 잎으로 바꿀 때의 '유효 α' = [R(t) − R(T_t)] / (|leaves(T_t)| − 1) 가 가장 작은 가지부터 순서대로 잘라냄 → 중첩된 부분트리 열 $T_0 \supset T_1 \supset \dots \supset \{root\}$
- 정리(Breiman): 각 α 에 대해 $R_\alpha$ 를 최소화하는 최소 부분트리가 유일하게 존재하며, 위 열이 그것들을 모두 포함한다 (weakest-link pruning)
- α 는 교차검증으로 선택. scikit-learn: cost_complexity_pruning_path(X, y) 로 α 열을 얻고 DecisionTreeClassifier(ccp_alpha=α) 로 학습 (그림: α↑ → 잎↓, 테스트 정확도는 중간 α 에서 최대)
- C4.5 는 잎의 오차율 상한(비관적 오차, pessimistic error)으로 가지치기 — 검증 데이터 불필요

# 이론 (2) — 표현력, 축 정렬 편향, 최적 트리의 난이도

## 표현력 (만능 근사)

- 잎을 충분히 늘리면 조각별 상수 함수로 임의의 연속 함수(또는 경계)를 원하는 정밀도로 근사 가능 — 그러나 필요한 잎 수가 차원에 대해 지수적
- n 개 훈련점을 (중복 없으면) 항상 100% 맞출 수 있음 → VC 차원이 잎 수와 함께 무한히 커짐 → 복잡도 제어 없이는 일반화 보장 없음

## 축 정렬(axis-aligned) 편향

- 분할은 항상 한 속성의 임계값 → 결정 경계는 축에 평행한 계단. 대각선 경계 $x_1 + x_2 > c$ 는 무수한 분할이 필요 (20쪽 그림 참고)
- 대안: 사선 트리(oblique tree, OC1), 회전 포레스트, 특징 조합($x_1+x_2$) 추가, 또는 앙상블로 평균내어 매끄럽게
- 역으로: 특징이 의미 있는 축이라면(임상 변수 임계값) 축 정렬은 해석에 유리

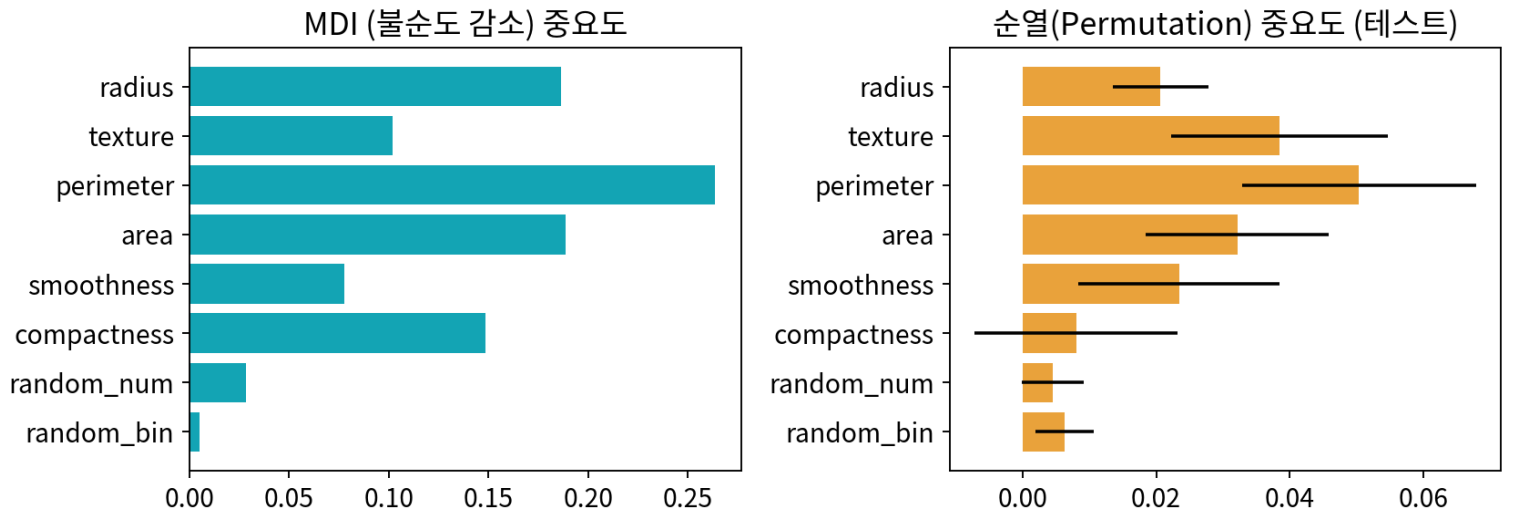
## 최적 트리는 어렵다

- 주어진 데이터와 일치하는 최소 크기의 의사결정나무를 찾는 문제는 NP-완전 (Hyafil & Rivest 1976) → 탐욕적 top-down 이 표준
- 최근: 정수계획/동적계획 기반 최적 희소 트리 (GOSDT, Lin et al. 2020; McTavish et al. 2022; DL8.5) — 얕은 깊이(≤5)·이진 특징에서 실용화, 해석 가능 모델 연구의 핵심 도구

## 일관성 (consistency)

- 잎의 크기가 n 과 함께 커지되($\to \infty$) 잎의 지름은 0 으로 가면, 트리 추정량은 베이즈 오차로 수렴 (Devroye, Györfi & Lugosi 1996, Thm 20.x)
- 랜덤 포레스트의 일관성·수렴률 증명 (Biau 2012; Scornet, Biau & Vert 2015) — 가법 모델에서 Breiman 의 원본 RF 가 일관됨
- 실무 함의: min_samples_leaf 를 n 과 함께 키우는 것이 이론적으로 정당

# 특성 중요도 — MDI 와 순열 중요도



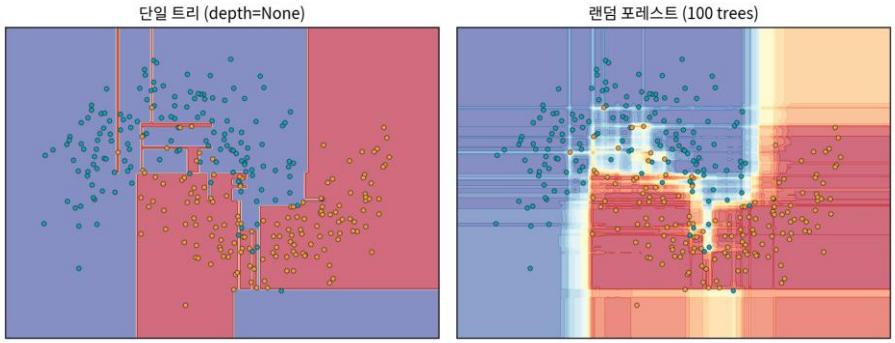

**MDI (Mean Decrease in Impurity) — feature_importances_**

- 각 속성이 만든 분할들의 (가중) 불순도 감소량을 트리 전체(포레스트면 평균)에 대해 합산. 학습 중 공짜로 얻어지고 합이 1
  - 편향: 고유값이 많은 연속형·고카디널리티 속성이 과대평가됨 — 그림에서 순수 잡음인 random_num 이 이진 잡음 random_bin 보다 높은 MDI 를 받음 (Strobl et al. 2007). 훈련 데이터 기준이라 과적합된 분할도 '중요' 로 집계

**순열 중요도 (Permutation importance, Breiman 2001)**

- 테스트 데이터에서 속성 j 의 값을 무작위로 섞었을 때의 성능 하락. 모델 무관(model-agnostic), 검증 데이터 기준 → 잡음 속성은 ≈ 0
  - 주의: 상관된 속성끼리는 서로 대체 가능하여 둘 다 낮게 나옴 → 상관 군집 단위로 함께 섞거나 조건부 순열 사용. scikit-learn: sklearn.inspection.permutation_importance

# 앙상블 (1) — 배깅과 랜덤 포레스트



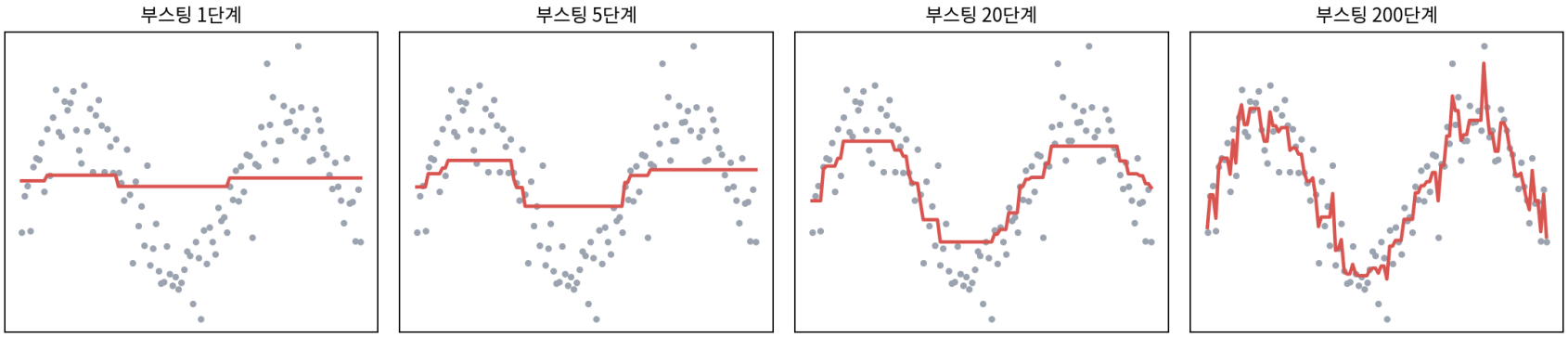

## 랜덤 포레스트 (Breiman 2001)

- B 개의 부트스트랩 표본 각각에 트리를 완전히 키움 (가지치기 없음)
- 각 분할에서 전체 d 개 중 무작위 m 개 속성만 후보 (분류 m=√d, 회귀 m=d/3 권장)
- 예측 = 다수결 / 평균. OOB(out-of-bag) 샘플로 검증 오차를 공짜로 추정

$$\mathrm{Var}[\,\text{평균}\,] = \rho\,\sigma^2 + (1-\rho)\,\sigma^2 / B \qquad (\rho\text{: 트리 간 상관, } \sigma^2\text{: 단일 트리 분산})$$

- B → ∞ 이면 둘째 항은 사라지고 ρσ² 만 남음 → 분산 감소의 한계는 트리 간 상관 ρ 가 결정. 속성 무작위 추출(m < d)은 ρ 를 낮추기 위한 장치 — 이것이 배깅과 랜덤 포레스트의 차이
- 편향은 거의 그대로(완전히 자란 트리 = 낮은 편향)이므로 RF = '저편향 고분산 학습기의 분산 축소'. 트리 수 B 를 늘려도 과적합하지 않음 (수렴할 뿐)
- Extra-Trees (Geurts 2006): 임계값까지 무작위로 뽑아 ρ 를 더 낮춤. 더 빠르고 잡음에 강건
- Isolation Forest (Liu 2008): 무작위 분할로 점을 고립시키는 데 필요한 깊이가 짧을수록 이상치 — 비지도 이상탐지의 표준 (9주차 군집화의 이상치 전처리와 연결)
- 장점: 하이퍼파라미터에 둔감, 병렬 학습, 결측(≥1.4)·혼합형 처리. 단점: 메모리·추론 속도, 외삽 불가, 해석은 중요도·SHAP 에 의존

# 앙상블 (2) — 그래디언트 부스팅




$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x), \quad h_m \approx \operatorname{argmin}_h \sum_i \left[ -\partial L(y_i, F)/\partial F \,|_{F=F_{m-1}(x_i)} - h(x_i) \right]^2$$

**함수 공간의 경사하강 (Friedman 2001)**

- 현재 모델 $F_{m-1}$ 의 손실 기울기(제곱손실이면 잔차 $y - F$)를 얕은 회귀나무 $h_m$ 으로 근사하고, 학습률 $\eta$(0.01~0.3)로 더해 나감. 그림: 1 → 5 → 20 → 200 단계에서 잔차가 점점 흡수됨
- 약한 학습기(depth 2~6)를 순차적으로 더해 편향을 줄임 — RF 와 정반대 전략. 단계 수 M 이 크면 과적합 → 조기 종료(validation) 필수
- AdaBoost (Freund & Schapire 1997) 는 지수 손실에 대한 특수 경우. 로지스틱 손실이면 LogitBoost / 현대 GBM

**정규화 장치**

- 학습률 $\eta$(shrinkage), 부분표본(subsample, 확률적 GB), 트리 깊이·잎 수·min_child_weight, L1/L2 잎 가중치 벌점(XGBoost), 조기 종료

# 앙상블 (3) — XGBoost 의 2차 근사와 분할 이득

$$\mathrm{Obj}^{(t)} \approx \Sigma_i \left[ g_i f_t(x_i) + \tfrac{1}{2} h_i f_t(x_i)^2 \right] + \gamma T + \tfrac{1}{2}\lambda \Sigma_j w_j^2, \quad g_i = \partial L/\partial F,\ h_i = \partial^2 L/\partial F^2$$

- 손실을 2차 테일러 전개 (Newton 부스팅) → 잎 j 의 최적 가중치와 목적함수가 닫힌 형태로 구해짐 (Chen & Guestrin 2016):

$$w^*_j = -G_j / (H_j + \lambda), \quad \mathrm{Obj}^* = -\tfrac{1}{2} \Sigma_j G_j^2 / (H_j + \lambda) + \gamma T, \quad G_j = \Sigma_{i \in \text{leaf } j} g_i,\ H_j = \Sigma_{i \in \text{leaf } j} h_i$$

$$\mathrm{Gain}(\text{분할}) = \tfrac{1}{2} \left[ G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - (G_L+G_R)^2/(H_L+H_R+\lambda) \right] - \gamma$$

**읽는 법**

- $G^2/(H+\lambda)$ 는 '유사도 점수'. 분할 이득이 $\gamma$ 보다 작으면 분할하지 않음 → 사전 가지치기가 목적함수에 내장. $\lambda$ 는 잎 가중치를 0 으로 당기는 L2 정규화
- 제곱손실이면 $g_i = F(x_i) - y_i$ (잔차의 음수), $h_i = 1$ → 정보이득·지니와 같은 형태의 '불순도 감소' 기준으로 환원됨 — Part 2 의 정보이득과 같은 틀

**시스템 측면 (표 형태 데이터에서 XGBoost 가 지배적인 이유)**

- 가중 분위수 스케치로 후보 임계값 근사, 결측치의 기본 방향(sparsity-aware) 학습, 열 블록 캐시·병렬화, GPU 히스토그램 학습

# 앙상블 (4) — 부스팅 라이브러리 비교

| | XGBoost (2016) | LightGBM (2017) | CatBoost (2018) | sklearn HistGB (1.0+) |
|---|---|---|---|---|
| **트리 성장** | level-wise (depth-wise), hist 옵션 | leaf-wise (최대 손실 감소 잎 확장) → 빠르고 정확하나 과적합 주의 (num_leaves) | 대칭(oblivious) 트리 — 같은 깊이에 같은 분할, 추론 매우 빠름 | level-wise, 256-bin 히스토그램 |
| **연속형 처리** | 정확/근사 분위수, hist | 히스토그램(bin) + GOSS(기울기 기반 표본) + EFB(희소 특징 묶기) | 히스토그램 | 히스토그램 |
| **범주형** | enable_categorical (1.6+) | 네이티브 (categorical_feature) | 네이티브, ordered target statistics — 타깃 누수 방지가 핵심 설계 | categorical_features |
| **결측치** | 기본 방향 학습 | 기본 방향 학습 | min/max 처리 옵션 | 기본 방향 학습 |
| **강점** | 생태계·GPU·안정성 | 대규모·고차원 속도 | 범주형 많은 데이터, 기본값만으로 강함 | 의존성 없이 sklearn 파이프라인 |

- 벤치마크 결론(Grinsztajn et al. NeurIPS 2022; McElfresh et al. 2023): 중간 규모($10^3$~$10^5$) 표 형태 데이터에서 GBDT 계열이 딥러닝(MLP·Transformer)을 평균적으로 이김 — 특히 비정보 특징·불규칙 함수·회전 비불변성이 이유
- 실무 기본 레시피: HistGradientBoosting 또는 LightGBM 으로 시작 → 학습률 0.05, 조기 종료, num_leaves/depth·min_child·subsample·정규화를 Optuna 로 탐색 → 검증 지표(AUROC/AUPRC) 로 선택

# 하이퍼파라미터 튜닝과 검증 설계

| 모델 | 우선 조정 | 전형적 범위 | 비고 |
|---|---|---|---|
| **DecisionTree** | ccp_alpha, max_depth, min_samples_leaf | α: CCP 경로에서 탐색 · depth 2–10 · leaf 1–50 | 단일 트리는 α 하나만 잘 잡아도 충분한 경우가 많음 |
| **RandomForest** | max_features, min_samples_leaf, n_estimators | √d 또는 0.3–0.7 · leaf 1–20 · trees 300–1000 | trees 는 많을수록 안정(과적합 없음), OOB 로 빠른 점검 |
| **GBDT (XGB/LGBM)** | learning_rate, n_estimators(조기종료), depth/num_leaves, subsample, colsample, reg_lambda | η 0.01–0.1 · depth 3–8 · num_leaves 15–255 · sub/col 0.6–1.0 · λ 0–10 | η 를 낮추고 조기 종료로 M 을 결정하는 것이 기본 전략 |
| **CatBoost** | depth, l2_leaf_reg, learning_rate | depth 4–10 · l2 1–10 | 기본값만으로도 강해 탐색 예산을 아낄 수 있음 |

## 탐색 방법

- 그리드 탐색은 차원이 커지면 비효율 → 무작위 탐색 (Bergstra & Bengio 2012)이 같은 예산에서 더 좋음
- 베이즈 최적화(TPE): Optuna — study.optimize(objective, n_trials=100), 조기 종료 가지치기(pruner) 와 함께 사용
- sklearn: HalvingRandomSearchCV (successive halving) 로 예산 절감

## 검증 설계 — 누수를 막는 규칙

- 튜닝은 훈련 데이터의 교차검증 안에서만. 테스트는 마지막에 한 번
- 전처리(인코딩·대치·스케일링)는 Pipeline 안에 넣어 폴드마다 다시 적합
- 시간 순서가 있으면 TimeSeriesSplit, 기관별 일반화를 보려면 GroupKFold(기관)
- 중첩 교차검증(nested CV)은 성능 추정의 편향을 없애지만 비용이 큼 — 논문용 성능 보고에 권장

# 표 형태 데이터: 트리 vs 딥러닝 — 최신 동향 (2022–2026)

## 왜 트리가 강한가 (Grinsztajn 2022)

- 표 형태 데이터의 목표 함수는 불규칙(비매끄러움)하고, 신경망은 매끄러운 해에 편향됨
- 비정보(잡음) 특징이 많을 때 트리는 무시하지만 MLP 는 영향받음
- 데이터가 회전 불변이 아님(각 열이 의미를 가짐) — 축 정렬 분할이 오히려 유리한 귀납 편향

## 딥러닝의 반격

- TabPFN (Hollmann et al., Nature 2025): 합성 데이터로 사전학습된 Transformer 가 소규모(≤10k 행) 표 데이터에서 문맥 내 학습(in-context)으로 GBDT 를 능가, 학습 없이 수 초 만에 예측
- TabM (2025), FT-Transformer, TabR 등 — 대규모에서는 GBDT 와 비슷하거나 앙상블로 상보적
- LLM 기반 표 데이터 추론은 아직 GBDT 에 못 미침

**2026 시점의 실무 권고**

- n < 10k: TabPFN(v2)과 GBDT 를 모두 시도하고 검증으로 선택. n ≥ 10k: GBDT(LightGBM/XGBoost/CatBoost)가 여전히 기본값. 매우 큰 n·다중 모달(영상+표)이면 신경망
- 트리와 신경망의 앙상블(스태킹)은 거의 항상 소폭 개선 — 대회(Kaggle)·임상 예측 논문에서 표준
- 해석·규제가 중요한 의료 응용은 얕은 트리·GAM(EBM: Explainable Boosting Machine, Microsoft InterpretML)·GBDT+SHAP 조합이 현실적 선택

**연구 질문**

- 결측·이질성·분포 이동(병원 간)에 대한 강건성, 공정성 제약 하의 트리 학습, 인과 추론용 트리(causal forest, Wager & Athey 2018)

# 트리 모델의 확장

## 다중 클래스 · 다중 출력

- 트리는 잎에 클래스별 개수를 두므로 다중 클래스가 자연스러움 (one-vs-rest 불필요)
- 다중 출력 회귀·분류: 잎에 벡터를 두어 한 트리로 여러 타깃 예측 (DecisionTree/RandomForest 는 지원, GBDT 는 타깃별 모델)
- 다중 라벨: 라벨 간 상관을 살리려면 classifier chain 또는 다중 출력 트리

## 비용 민감 · 생존 분석

- 오분류 비용이 비대칭이면 class_weight / sample_weight 로 불순도 계산에 반영 — 임계값 조정과 병행
- 생존 자료(중도절단): 생존 트리·랜덤 생존 포레스트 (Ishwaran 2008), 로그 순위 검정 기반 분할 — scikit-survival
- XGBoost: survival:cox, survival:aft 목적함수 지원 (외상 레지스트리 생존 분석에 활용)

## 온라인 · 스트리밍 · 대규모

- Hoeffding 트리(VFDT, Domingos & Hulten 2000): 한 번씩만 보는 스트림에서 Hoeffding 한계로 분할 시점을 통계적으로 결정 — river 패키지
- 분포 이동 감지(ADWIN)와 결합한 적응형 랜덤 포레스트
- 대규모: 히스토그램 기반 학습, GPU(XGBoost device='cuda', cuML), 분산(Spark)

## 다른 모델과의 관계

- 규칙 모델: RuleFit(트리에서 규칙을 뽑아 라쏘), 결정 리스트 — 해석 가능 모델
- GAM 계열: EBM(Explainable Boosting Machine)은 특징별 얕은 부스팅 트리의 합 → 형태 함수를 그림으로 제시
- 신경망과의 연결: soft decision tree, NODE, 그리고 트리 앙상블을 신경망으로 증류(distillation)

# 해석 — SHAP (TreeSHAP) 과 부분의존도

## SHAP (Lundberg & Lee 2017)

$\varphi_j = \Sigma_{S \subseteq F \setminus \{j\}} [|S|!(|F|-|S|-1)!/|F|!] \cdot [f(S \cup \{j\}) - f(S)]$

- 협력 게임의 Shapley 값: 각 특징이 개별 예측을 기준값에서 얼마나 밀었는지 공정하게 배분 (효율성·대칭성·가법성 공리)
- 일반 모델은 지수 시간 → TreeSHAP (Lundberg 2020) 은 트리 구조를 이용해 $O(TLD^2)$ 다항 시간에 정확 계산 — 트리 모델의 큰 실무 이점
- shap.TreeExplainer(model).shap_values(X)

## 시각화와 활용

- waterfall / force plot: 환자 한 명의 예측 근거 (국소 설명)
- beeswarm summary: 전역 중요도 + 방향 (값이 클 때 위험↑?)
- dependence plot: 특징–SHAP 관계와 상호작용 (비선형·임계값 확인)
- 부분의존도(PDP)·ICE: 다른 특징을 고정하고 한 특징을 변화시킬 때 평균 예측 — 상관된 특징에서는 비현실적 조합을 만들 수 있어 ALE 권장

**주의 — 설명 ≠ 인과**

- SHAP 은 '모델이 무엇을 사용했는가' 를 말하지 '무엇이 결과를 일으켰는가' 를 말하지 않음. 상관된 특징 간 기여 배분은 임의성이 있음(interventional vs observational SHAP)
- 임상 보고 시: 전역 SHAP 순위 + 대표 환자의 국소 설명 + 임계값(dependence plot)을 제시하고, 알려진 임상 지식과의 정합성으로 타당성을 논의
- 규칙 추출: 단일 트리는 그 자체가 설명. 포레스트/GBDT 는 SHAP, 또는 얕은 트리로 근사(distillation)·규칙 리스트(RuleFit) 로 요약

# 불균형 · 캘리브레이션 · 임계값

**클래스 불균형 (의료: 사망 2%, 중증 5% 등)**

- class_weight='balanced' (트리·RF) 또는 scale_pos_weight (XGBoost): 소수 클래스 오분류에 큰 비용 → 분할 기준의 가중 불순도가 바뀜
- 리샘플링(SMOTE 등)은 트리에서 확률을 왜곡하므로 신중히. 대신 지표를 AUPRC·F-β·비용 기반으로 바꾸고 임계값을 조정하는 것이 우선

**확률 캘리브레이션**

- 단일 트리의 잎 확률은 거칠고 극단적(0/1), RF 는 중앙으로 몰리는 경향(과소확신), GBDT 는 비교적 잘 맞으나 조기 종료·정규화에 따라 달라짐
- 검증 데이터로 Platt(시그모이드)·등장회귀(isotonic) 보정: CalibratedClassifierCV. 캘리브레이션 곡선과 Brier 점수로 확인

**임계값과 운영점**

- 0.5 는 기본값일 뿐. 민감도 목표(예: 중증 외상 과소분류 ≤ 5%)나 비용에 맞춰 검증 데이터에서 임계값 선택 → 테스트에 고정 적용. TunedThresholdClassifierCV (sklearn ≥ 1.5)
- 운영점을 바꾸면 하위집단(연령·성별) 간 민감도 차이가 달라짐 — 공정성 점검 필요

# 의료 · 보건 데이터 응용

## 임상 결정 규칙 = 얕은 트리

- Ottawa 발목 규칙, PECARN 소아 두부외상 규칙(재귀 분할로 도출, Lancet 2009), 캐나다 CT 두부 규칙
- CART 로 만든 규칙은 '민감도 우선' 으로 학습(비용 가중), 외부 검증 후 지침에 채택
- 응급 트리아지: 활력징후 임계값의 순차 질문 → 트리 구조 그 자체

## 예측 모델링 (GBDT)

- 외상 사망·중증도 예측(레지스트리 데이터)에서 XGBoost/LightGBM 이 로지스틱(TRISS) 대비 AUROC 개선을 흔히 보고
- EHR 기반 조기경보(패혈증, 악화), 재입원, 재원일수
- 필수: 시간적 외부 검증, 캘리브레이션, 결측 패턴의 정보 누수 점검, TRIPOD+AI 보고 지침

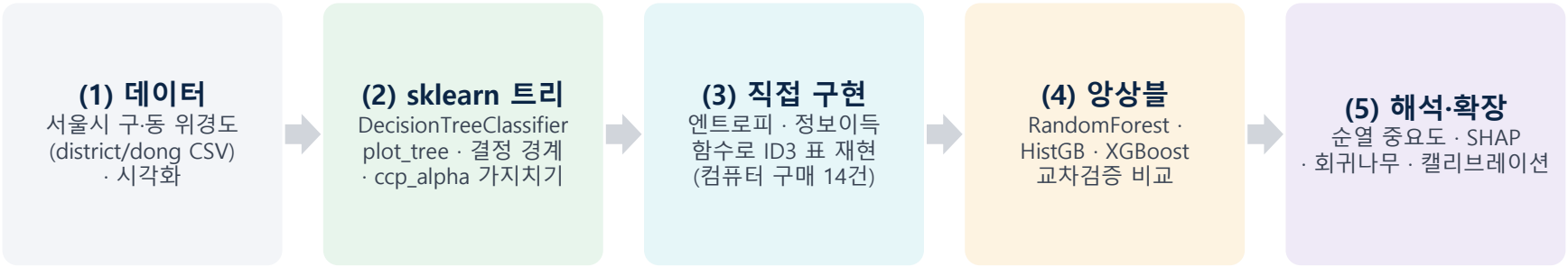
## 인과 · 이질적 치료효과

- Causal forest (Wager & Athey 2018): 잎 안에서 처치–대조 차이를 추정하는 '정직한(honest)' 트리 — 누가 치료 이득이 큰가
- 정책 트리(policy tree): 해석 가능한 치료 배정 규칙
- 관찰 연구에서는 교란 통제가 전제 — 방법이 인과를 보장하지 않음

## 보고 시 필수 항목

- 데이터 분할(시간/기관), 결측 처리, 하이퍼파라미터 탐색 범위와 방법
- 성능: AUROC·AUPRC·캘리브레이션·결정곡선(DCA), 하위집단별
- 해석: SHAP 전역·국소, 임상 지식과의 정합성
- 코드·시드·모델 파일 공개, 외부 검증 계획

# 실습 개요 — 9주차_의사결정트리_2026.ipynb




**환경**

- Python 3.11+, scikit-learn ≥ 1.7, xgboost ≥ 2.0, shap ≥ 0.45, matplotlib. graphviz 대신 sklearn.tree.plot_tree 사용(설치 불필요). Colab 에서 바로 실행
- 기존 노트북과의 차이: LabelEncoder 를 y 에 직접 사용(1D) 또는 문자열 y 그대로 fit 가능, export_graphviz → plot_tree, 데이터 로드 시 로컬 경로/URL 모두 지원, 앙상블·해석 절 추가

**과제 제안 (대학원)**

- ① 정보이득 ≥ 0 을 100개 무작위 데이터셋에서 수치로 확인 · ② ccp_alpha 경로와 교차검증으로 최적 α 선택 후 테스트 · ③ 본인 연구 데이터에 GBDT + SHAP + 캘리브레이션 파이프라인 적용 · ④ 지니 vs 엔트로피 트리가 다른 경우의 비율 측정

# 코드 (1) — 트리 학습과 시각화 (sklearn 1.7+)

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
from sklearn.metrics import accuracy_score, classification_report

train = pd.read_csv("district_dict_list.csv"); test = pd.read_csv("dong_dict_list.csv")
X_train, y_train = train[["longitude", "latitude"]], train["label"]      # y 는 1차원 Series (문자열 OK)
X_test,  y_test  = test[["longitude", "latitude"]],  test["label"]

clf = DecisionTreeClassifier(criterion="gini", max_depth=4, min_samples_leaf=2, random_state=0)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
print("accuracy:", accuracy_score(y_test, pred)); print(classification_report(y_test, pred))

fig, ax = plt.subplots(figsize=(14, 6))                                    # graphviz 불필요
plot_tree(clf, feature_names=["longitude", "latitude"], class_names=clf.classes_,
          filled=True, rounded=True, impurity=True, fontsize=8, ax=ax)
print(export_text(clf, feature_names=["longitude", "latitude"]))          # 텍스트 규칙

# 결정 경계 (sklearn >= 1.1)
from sklearn.inspection import DecisionBoundaryDisplay
DecisionBoundaryDisplay.from_estimator(clf, X_train, alpha=0.3, response_method="predict")
plt.scatter(X_train.longitude, X_train.latitude, c=pd.factorize(y_train)[0], edgecolor="k", s=20)
```

*clf.tree_ 로 내부 배열(children_left, feature, threshold, impurity, value) 접근 가능 — 트리 구조를 직접 순회해 규칙을 뽑거나 잎 통계를 계산할 때 사용.*

# 코드 (2) — 엔트로피 · 정보이득 직접 구현 (ID3 표 재현)

```
df = pd.read_csv("https://raw.githubusercontent.com/AugustLONG/ML01/master/01decisiontree/AllElectronics.csv").drop("RID",
axis=1)
TARGET = "class_buys_computer"

def entropy(y):                                  # H = -Σ p log2 p  (p=0 항은 제외)
    p = y.value_counts(normalize=True).to_numpy()
    p = p[p > 0]
    return float(-(p * np.log2(p)).sum())

def cond_entropy(df, attr):                      # H_A(D) = Σ_v |D_v|/|D| · H(D_v)
    n = len(df)
    return sum(len(sub) / n * entropy(sub[TARGET]) for _, sub in df.groupby(attr))

def info_gain(df, attr):
    return entropy(df[TARGET]) - cond_entropy(df, attr)

print("H(D) =", round(entropy(df[TARGET]), 3))                              # 0.940
gains = {a: round(info_gain(df, a), 3) for a in df.columns if a != TARGET}
print(gains)                                      # {'age': 0.247, 'income': 0.029, 'student': 0.151, 'credit_rating':
0.048}
best = max(gains, key=gains.get); print("root split:", best)

for v, sub in df.groupby(best):                   # 자식 노드에서 재귀 계산 (청소년/중년 → 학생/신용)
    g = {a: round(info_gain(sub, a), 3) for a in sub.columns if a not in (TARGET, best)}
    print(v, "H =", round(entropy(sub[TARGET]), 3), g)

# 검증: 정보이득은 항상 >= 0  (이론 1)
assert all(v >= -1e-12 for v in gains.values())
```

*sklearn 으로 같은 트리를 얻으려면 DecisionTreeClassifier(criterion='entropy') 에 원-핫 인코딩한 X 를 넣는다 (CART 는 이진 분할이므로 모양은 조금 다름).*

# 코드 (3) — 가지치기와 앙상블 비교

```
from sklearn.model_selection import cross_val_score, StratifiedKFold, train_test_split
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from xgboost import XGBClassifier

# (a) 비용-복잡도 가지치기: alpha 경로 → 교차검증으로 선택
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(X_train, y_train)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
scores = [cross_val_score(DecisionTreeClassifier(ccp_alpha=a, random_state=0), X_train, y_train, cv=cv).mean()
          for a in path.ccp_alphas]
best_alpha = path.ccp_alphas[int(np.argmax(scores))]
pruned = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=0).fit(X_train, y_train)
print("best alpha:", best_alpha, "leaves:", pruned.get_n_leaves(), "test acc:", pruned.score(X_test, y_test))

# (b) 앙상블 비교 (y 를 정수로: XGBoost 는 정수 라벨 필요)
y_tr_i, classes = pd.factorize(y_train); y_te_i = pd.Categorical(y_test, categories=classes).codes
models = {
  "tree(pruned)": pruned,
  "random_forest": RandomForestClassifier(500, max_features="sqrt", oob_score=True, n_jobs=-1, random_state=0),
  "hist_gb":       HistGradientBoostingClassifier(learning_rate=0.05, max_iter=500, early_stopping=True, random_state=0),
  "xgboost":       XGBClassifier(n_estimators=500, learning_rate=0.05, max_depth=4, subsample=0.8,
                                 colsample_bytree=0.8, reg_lambda=1.0, early_stopping_rounds=30, random_state=0),
}
for name, m in models.items():
    if name == "xgboost":
        Xa, Xv, ya, yv = train_test_split(X_train, y_tr_i, test_size=0.2, stratify=y_tr_i, random_state=0)
        m.fit(Xa, ya, eval_set=[(Xv, yv)], verbose=False); acc = (m.predict(X_test) == y_te_i).mean()
    elif name == "tree(pruned)": acc = m.score(X_test, y_test)
    else: m.fit(X_train, y_tr_i); acc = m.score(X_test, y_te_i)
    print(f"{name:14s} test acc = {acc:.3f}")
print("RF OOB score:", models["random_forest"].oob_score_)
```

# 코드 (4) — 해석: 순열 중요도와 SHAP

```
from sklearn.inspection import permutation_importance
import shap

rf = models["random_forest"]
pi = permutation_importance(rf, X_test, y_te_i, n_repeats=30, random_state=0, n_jobs=-1)
print(pd.DataFrame({"MDI": rf.feature_importances_, "perm_mean": pi.importances_mean,
                    "perm_std": pi.importances_std}, index=X_test.columns).sort_values("perm_mean", ascending=False))

xgb = models["xgboost"]
explainer = shap.TreeExplainer(xgb)                       # TreeSHAP: 다항 시간 정확 계산
sv = explainer(X_test)                                    # shap.Explanation (다중 클래스면 [n, d, K])
shap.plots.beeswarm(sv[:, :, 0])                          # 클래스 0 에 대한 전역 요약
shap.plots.waterfall(sv[0, :, 0])                         # 첫 번째 관측치의 국소 설명
shap.plots.scatter(sv[:, "latitude", 0])                  # 위도-SHAP 의존도 (임계값 확인)

# 회귀나무 (연차·안타 → 연봉 예제)
from sklearn.tree import DecisionTreeRegressor
Xr = np.array([[2,20],[3,26],[4,23],[5,35],[6,30],[7,52],[8,38],[9,40],[11,60],[12,50],[13,90],[14,80],[15,92]])
yr = np.array([27,22,27,27,19,41,22,24,88,87,81,85,88])
rt = DecisionTreeRegressor(max_depth=2).fit(Xr, yr)
print(export_text(rt, feature_names=["years", "hits"]))  # 분할 임계값과 잎 평균 확인
print("새 선수 (연차 5, 안타 35) 예측 연봉:", rt.predict([[5, 35]]))
```

- 캘리브레이션: from sklearn.calibration import CalibratedClassifierCV, calibration_curve — 검증 폴드로 isotonic 보정 후 Brier 점수 비교
- 임계값 조정: from sklearn.model_selection import TunedThresholdClassifierCV (sklearn ≥ 1.5) 로 목표 민감도에 맞는 운영점 선택

# 흔한 실수와 체크리스트

## 자주 하는 실수

- max_depth=None 기본값으로 단일 트리를 쓰고 테스트 성능에 놀람
- 훈련 정확도 100% 를 성과로 보고
- feature_importances_(MDI) 만으로 '중요 변수' 를 주장
- 고카디널리티 ID·날짜 열을 넣어 정보이득이 최대가 되게 함 (누수)
- 타깃 인코딩을 전체 데이터로 계산 (CatBoost 의 ordered TS 가 막는 누수)
- 조기 종료 없이 n_estimators 만 키움
- SHAP 을 인과 효과로 해석
- 불균형 데이터에서 accuracy 로 모델 선택
- 시간 순서 있는 데이터를 무작위 분할

## 제출 전 체크리스트

- ☐ 분할(시간/기관)과 누수 점검을 했는가
- ☐ 단일 트리는 ccp_alpha 또는 깊이를 교차검증으로 정했는가
- ☐ GBDT 는 학습률·조기 종료·정규화를 탐색했는가
- ☐ AUROC·AUPRC·캘리브레이션·결정곡선을 보고했는가
- ☐ 순열 중요도/SHAP 과 임상 지식의 정합성을 논의했는가
- ☐ 운영 임계값의 근거와 하위집단 성능을 제시했는가
- ☐ 로지스틱 회귀 등 단순 기준 모델과 비교했는가
- ☐ 코드·시드·하이퍼파라미터·모델 파일을 공개했는가

# 핵심 정리

### 구조 · 분할 기준

- 트리 = 축 정렬 재귀 분할 + 영역별 상수
- 엔트로피 $H = -\Sigma\, p \log p$, 지니 = $1 - \Sigma\, p^2$
- 정보이득 = $H(D) - H_A(D) = I(Y;A) \geq 0$
- 편향 → 이득비, 이진 분할(CART)
- 회귀나무: 잔차제곱합 최소, 잎 = 평균

### 이론 · 과적합

- 탐욕 성장은 스스로 멈추지 않음 → 가지치기
- CCP: $R(T) + \alpha|T|$, weakest-link, $\alpha$ 는 CV
- 고분산·저편향 → 배깅/RF 로 분산 축소 ($\rho\sigma^2$)
- 부스팅: 함수 공간 경사하강, XGBoost 2차 근사
- 최적 트리 NP-완전, 축 정렬 편향, 일관성

### 실무 · 최신

- GBDT 가 표 데이터 기본값, TabPFN 은 소규모에서 경쟁
- 해석: 순열 중요도, TreeSHAP (다항 시간), PDP/ALE
- 불균형·캘리브레이션·운영 임계값·하위집단
- 의료: 임상 결정 규칙, 예측 모델, causal forest
- sklearn 1.7: NaN 지원, plot_tree, HistGB, TunedThreshold

# 참고문헌


- Quinlan, J. R. (1986). Induction of decision trees. Machine Learning 1. / Quinlan (1993). C4.5: Programs for Machine Learning.
- Breiman, Friedman, Olshen & Stone (1984). Classification and Regression Trees. / Breiman (2001). Random forests. Machine Learning 45.
- Hyafil & Rivest (1976). Constructing optimal binary decision trees is NP-complete. IPL. / Devroye, Györfi & Lugosi (1996). A Probabilistic Theory of Pattern Recognition.
- Friedman (2001). Greedy function approximation: A gradient boosting machine. Ann. Stat. / Chen & Guestrin (2016). XGBoost. KDD. / Ke et al. (2017). LightGBM. NeurIPS. / Prokhorenkova et al. (2018). CatBoost. NeurIPS.
- Geurts, Ernst & Wehenkel (2006). Extremely randomized trees. / Liu, Ting & Zhou (2008). Isolation forest. ICDM.
- Strobl et al. (2007). Bias in random forest variable importance measures. BMC Bioinformatics. / Lundberg & Lee (2017). SHAP. NeurIPS. / Lundberg et al. (2020). TreeSHAP. Nat. Mach. Intell.
- Grinsztajn, Oyallon & Varoquaux (2022). Why do tree-based models still outperform deep learning on tabular data? NeurIPS. / Hollmann et al. (2025). TabPFN. Nature 637.
- Lin, Zhong, Hu, Rudin & Seltzer (2020). Generalized and scalable optimal sparse decision trees (GOSDT). ICML. / McTavish et al. (2022). Fast sparse decision tree optimization. AAAI. / Wager & Athey (2018). Causal forests. JASA.
- Kuppermann et al. (2009). PECARN head trauma rules. Lancet. / Collins et al. (2024). TRIPOD+AI statement. BMJ.
- Hastie, Tibshirani & Friedman (2009). The Elements of Statistical Learning, Ch. 9–10, 15. / Han, Kamber & Pei (2011). Data Mining, Ch. 8 (AllElectronics 예제).
- scikit-learn 1.7 User Guide — 1.10 Decision Trees, 1.11 Ensembles; Pedregosa et al. (2011). JMLR.

# 질문 및 정리

Q & A

### 오늘의 핵심 정리

- 의사결정나무는 축 정렬 재귀 분할로 입력 공간을 나누고 영역마다 상수를 예측하는 해석 가능한 모델이다.
- 분할 속성은 정보이득(= 상호정보량 ≥ 0)·이득비·지니로 고르며, 컴퓨터 구매 예제에서 나이 → 학생/신용 순으로 트리가 완성된다.
- 탐욕 성장은 과적합을 만들므로 비용-복잡도 가지치기(ccp_alpha)와 깊이 제한이 필요하고, 트리의 고분산은 랜덤 포레스트가, 고편향은 부스팅이 보완한다.
- XGBoost·LightGBM·CatBoost 는 표 형태 데이터의 표준이며, TreeSHAP·순열 중요도로 해석하고 캘리브레이션·운영 임계값까지 보고해야 한다.
- 실습: scikit-learn 1.7 + XGBoost + SHAP 로 트리 → 가지치기 → 앙상블 → 해석 파이프라인을 완성한다.

성명 : 최승호 · 이메일 : jcn99250@naver.com